

Who’s Talking? Speech Bubble Attribution in Comics

Minnie Liang
MIT

minniejl@mit.edu

Ruoxi Qian
MIT

rqian@mit.edu

Abstract

Speech bubble attribution in manga—identifying which character speaks a line of dialogue—remains an open problem, as existing approaches rely on single-signal heuristics that break down in crowded panels. We propose a Detect-then-Match pipeline on Manga109 that pairs YOLOv8 detection with an XGBoost binary classifier over candidate body–bubble pairs. For detection, we ablate three YOLOv8 architectural variants targeting texture bias and find that in-domain fine-tuning already adapts the model to manga’s visual style, leaving these backbone modifications redundant. For attribution, we construct 10 spatial features per candidate pair with four feature subset configurations, three sample weighting schemes, two evaluation training regimes (and a partridge in a pear tree). Training on YOLO-predicted boxes rather than ground-truth coordinates, which aligns feature distributions between training and inference, produces the single largest accuracy gain (+5.2%). Feature importance analysis reveals that panel co-membership accounts for 82% of total gain, explaining both the strength of frame-aware heuristics and the negligible impact of weighting schemes under distribution-aligned training. Our best model achieves 65.1% accuracy on 22 volumes, outperforming the baseline by 7.6%. More generally, we find that distribution alignment between training and inference can be more impactful than architectural complexity, a lesson applicable to detection-classification systems beyond manga.

1. Introduction

Comics present a unique challenge for computer vision models. Unlike natural photographs, they consist of stylized images that are hand-drawn or digitally illustrated images, and can vary widely across different artists, genres, and geographic locations. Manga exemplifies this challenge: each page interleaves hand-drawn panels with speech text to convey a sequential narrative [2, 12]. A fundamental task in understanding these narratives is speech bubble attribution, or automatically determining which character is speaking the

text contained in each bubble.

Solving the task of speech bubble attribution requires addressing three different sub-problems: detecting characters, detecting speech bubbles, and correctly assigning each bubble to its speaker [20]. Each of these components has been studied individually, however, the full pipeline remains underexplored.

First, it remains unclear how well object detection architectures generalize to stylized, non-photorealistic images, where high object overlap and limited datasets have hindered progress compared to natural images [1, 18]. Understanding which architectural adaptations help, and also which are redundant, is valuable for guiding future work in this space. Furthermore, accurate speech bubble attribution is a prerequisite for several downstream applications in manga processing. Li et al. [14] demonstrate that character identification and speaker prediction are critical for tasks such as voice generation and automated translation of comics, where knowing which character speaks each line is necessary to maintain consistent tone and style across languages. Nguyen et al. [16] show that deep learning-based content extraction, such as including detection of panels, speech balloons, and characters, can enable content-based retrieval and search over large comic archives. However, a translation pipeline that cannot distinguish speakers is unreliable, and a retrieval system that indexes dialogue without speaker identity loses critical information.

Existing attribution pipelines decompose the problem into detection followed by attribution through heuristics such as spatial proximity or balloon-tail geometry. However, these rule-based methods often fail in crowded panels where characters are similarly close to a bubble. Meanwhile, YOLO adaptations for stylized images, such as the YOLOv5+XGBoost pipeline proposed by Ahmad et al. [1], have not kept pace with recent architectural advances: YOLOv8 (2023), YOLO11 (2024), and YOLOv26 (2026), which may offer improved feature extraction and better performance under domain shift [9, 10].

We propose a Detect-then-Match pipeline that addresses both gaps. We specifically investigate whether texture-bias modifications improve in-domain manga detection, and

whether spatial feature engineering and training distribution alignment improve attribution. In the first stage, YOLOv8s detects all character bodies and speech bubbles on each manga page. We evaluate whether architectural modifications to the backbone (InstanceNorm, style-robust C2f blocks) and neck (CBAM attention) improve detection, and apply XGBoost as a post-processing filter to suppress false positive detections.

In the second stage, every detected bubble is paired with every detected body on the same page, and an XGBoost binary classifier scores each pair using a set of 10 normalized spatial features derived from Manga109’s frame annotations. The highest-scoring body is selected as the predicted speaker. We evaluate two training regimes (ground-truth boxes vs. YOLO-predicted boxes for feature computation), three sample weighting schemes (uniform, page-difficulty, YOLO-confidence), and four feature set configurations to isolate each design decision’s contribution.

Our key contributions are:

- (i) A systematic detection ablation study evaluating three YOLOv8 architectural variants, establishing that in-domain fine-tuning renders texture-bias modifications redundant while neck-level attention provides only modest improvement.
- (ii) A demonstration that training distribution alignment (YOLO-matched vs. GT regimes) provides the largest accuracy gain (+5.2%) in the attribution pipeline, a finding with implications for any Detect-then-Match system. Our best model achieves 65.1% attribution accuracy on 22 held-out volumes.
- (iii) A feature analysis revealing that panel co-membership (`same_frame`) dominates all other attribution features, explaining both the strength of simple frame-aware heuristics and the ineffectiveness of sample weighting schemes.

2. Related Work

2.1. Object Detection in Comics

Standard CNNs are trained to process natural images and are texture-biased, meaning they recognize objects more by surface patterns rather than shape [6]. Since texture varies substantially between artists within manga art, shape is potentially a more important indicator of what an object. Techniques proposed to address texture bias include replacing BatchNorm with InstanceNorm to strip per-sample texture statistics [8], inserting Vision Transformer blocks for global shape reasoning, and adding channel attention mechanisms such as Squeeze-and-Excite blocks [7] and CBAM [22] to focus on semantically relevant features. We evaluate several of these techniques within the YOLOv8 architecture. Beyond single-model improvements, Ahmad and Schich [1] propose that combining a detector with a classi-

fier, specifically YOLOv5 with XGBoosting, can improve cross-domain detection on artwork images, a strategy that motivated our Detect-then-Match pipeline.

2.2. Speech Bubble Attribution

Early attribution approaches rely on a single spatial signal, whether balloon tail geometry [19] or nearest-neighbor heuristics [11]. However, tail detection is unreliable in manga as a nontrivial amount of speech bubbles lack tails. The nearest-edge baseline is a strong heuristic, however, it still relies on a single spatial signal and doesn’t account for structural cues like panel containment.

These limitations have motivated new approaches that combine multiple spatial signals into a single attribution decision. For instance, Li et al. frames attribution as a scene graph generation (SGG) problem [15], treating characters and text boxes as graph nodes to predict relational edges between them. Sachdeva et al. propose Magi [21], a model that detects text boxes and characters jointly, producing pairwise association scores for all text-character combinations via a transformer backbone. These existing methods implicitly enumerate candidate pairs to classify their relationships, similar to our approach, however their model architecture is entirely different, relying on learned reasoning networks rather than gradient-boosted trees over geometric features.

Closest to our work, Comic MTL [17] extends a Mask R-CNN model with a PairPool component that extracts features for candidate balloon-character pairs and feeds them into a binary classifier, predicting whether each pair is linked. This learned pairwise classification is most directly related to our approach, however, Comic MTL learns pair representations from a shared detection backbone that is trained end-to-end, whereas our method decouples detection from classification.

3. Dataset

3.1. Manga109 and Manga109Dialog

Manga109 [5] contains 10,602 pages across 109 volumes spanning diverse genres and artistic styles. The base annotation provides bounding boxes for four categories: body, face, text, and frame (panel boundary). We pair this dataset with the Manga109Dialog [13], which adds speaker-to-text links mapping each annotated speech bubble to its speaker body by annotation ID. We use only these human-annotated links as supervision.

3.2. Training and Validation

We split the dataset at the book level to prevent information leakage from artist style: the first 87 volumes form the training set (8,525 images), and the final 22 volumes form the held-out validation set (2,077 images). Because each

manga volume has a distinct artistic style, page layout conventions, and character designs, a book-level split ensures the model is evaluated on visual styles it has never seen during training.

3.3. Conversion into Usable Formats

Manga109 stores annotations as XML with absolute pixel coordinates (xmin, ymin, xmax, ymax) organized by book and page. YOLOv8 requires a different format: one text file per image, where each bounding box is represented as (class_id, center_x, center_y, width, height). We wrote a conversion script that iterates over all books and pages, extracts annotations for body (class 0) and text (class 1), and computes the normalized center coordinates and dimensions.

4. Methodology

Our system, Detect-then-Match, follows a two-stage architecture to improve speech bubble attribution. Figure 1 shows an example of our attribution pipeline applied to a sample page. As seen in Figure 2, in our architecture, in the detection stage, a fine-tuned YOLO model takes a raw manga page as input and localizes character bodies and text bubbles. In the attribution stage, we enumerate all candidate (text bubble, body) pairs on each page, extract a 10-dimensional geometric feature vector per pair, and score each pair with an XGBoost classifier. By decoupling detection from attribution, each stage can be independently tuned and evaluated. The following subsections describe each component in detail.

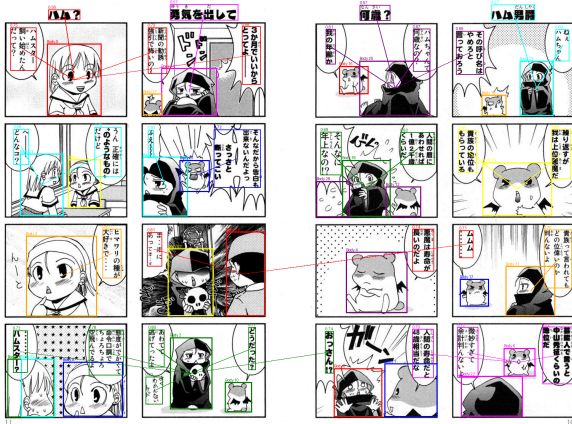


Figure 1. Attribution results on a sample page from the Manga109 dataset. Lines connect each text region to its predicted speaker, with attribution confidence scores displayed above each bubble.

4.1. Detection: Benchmarking YOLO Models

We benchmarked three YOLO architectures on Manga109 with identical training settings (20 epochs, AdamW opti-

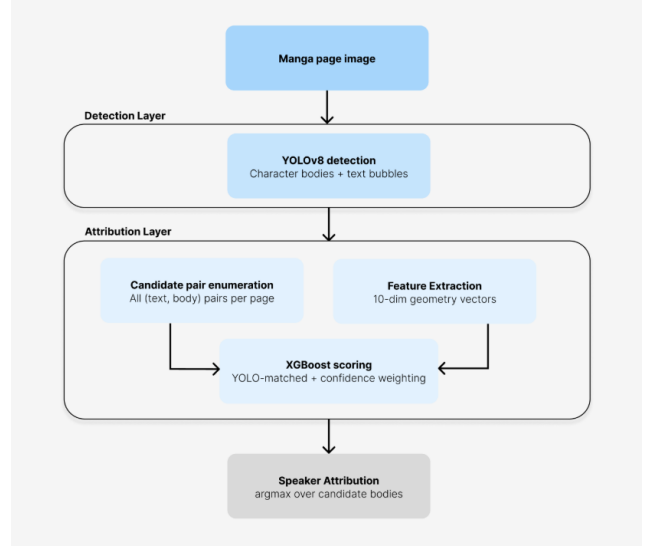


Figure 2. Entire Detect-then-Match pipeline, segmented by detection and then attribution layer

mizer, $lr=5 \times 10^{-4}$, batch size 8, image size 640). Standard object detection performance metrics were chosen: mAP50, mAP50-95, Precision, Recall, and F1. As shown in Table 1, YOLOv8s achieved the strongest overall performance and was selected as the baseline for all subsequent experiments.

Table 1. YOLO architecture comparison on Manga109. YOLOv8s achieves the highest mAP50 and F1 score at 0.93 and 0.88 respectively.

Model	mAP50	mAP50-95	Prec.	Rec.	F1
YOLOv8s	0.926	0.679	0.906	0.857	0.881
YOLOv11s	0.921	0.673	0.904	0.852	0.877
YOLOv26n	0.890	0.635	0.874	0.811	0.842

4.2. Detection: Architectural Changes

To identify whether architectural modifications could improve detection on manga’s stylized imagery, we evaluated three variants of YOLOv8s by swapping in different modules.

4.2.1. Shape-Biased Backbone

We made two changes to implement a more shape-biased model. First, we replaced BatchNorm with InstanceNorm in the first two convolutional layers to reduce sensitivity to instance-specific appearance statistics. While BatchNorm aggregates statistics across the batch and can reflect dataset-level biases, InstanceNorm normalizes each image independently, removing per-instance mean and variance [8]. This suppresses global style cues such as contrast and color dis-

tribution, encouraging the network to rely less on texture and more on shape-based features.

Our second change was to insert a lightweight self-attention block (ViTBlock) after the backbone. Unlike convolutions that operate on local patches, self-attention computes relationships between all spatial locations simultaneously, forcing global shape reasoning.

4.2.2. Style-Robust C2f Blocks

Texture bias accumulates through every stage of the backbone. To further reduce texture sensitivity in deeper layers, we modified the C2f (Cross Stage Partial) blocks, where most feature learning occurs.

Inside each C2f block, we kept the standard 1×1 convolution with BatchNorm for training stability, but replaced the 3×3 conv’s BatchNorm with InstanceNorm. The 3×3 conv is where spatial texture patterns are most strongly captured, so applying InstanceNorm there strips texture statistics at every residual step throughout the backbone.

Additionally, after C2f concatenates its split features, we added a Squeeze-and-Excitation block that globally average-pools the feature map and learns per-channel weights [7]. Channels responding to semantic content (shapes, boundaries) are upweighted while channels responding to texture are downweighted.

Neither modification produced meaningful improvement over the YOLOv8s baseline, so we pivoted to modifications in the model neck instead.

4.2.3. CBAM Attention in PANet Neck

Character sizing varies considerably across manga panels, with some bodies appearing large in the foreground and others small in the background. To address this, we added CBAM (Convolutional Block Attention Module)[?] after each neck C2f block. CBAM applies channel and spatial attention sequentially, helping the model focus on the right scale for each detection. For this experiment, we trained for both 20 and 40 epochs.

4.3. Detection: Post-Processing with XGBoosting

A consistent failure across all YOLO variants was a significant number of false positives. In an attempt to rectify this, we followed Ahmad et al. [1] and evaluated XGBoost as a post-processing filter for YOLO detections. Note that XGBoost does not modify YOLO’s weights; rather, it operates entirely on the metadata of each proposed detection and learns to suppress the types of detections that YOLO tends to get wrong.

For each detection proposed by YOLOv8s, we extracted 10 features: confidence score, normalized box width, height, area, aspect ratio, center x/y position, class, distance from image center, and box diagonal. We trained an XGBoost binary classifier [4] to predict whether each detection

is a true positive ($\text{IoU} \geq 0.5$ with a ground-truth box) or a false positive, using detections from the training set.

For the attribution stage, we selected the YOLO model with the fewest false negatives rather than the fewest false positives. This is because a false positive (a spurious detection) can be ignored by the downstream classifier, whereas a false negative (a missed character or speech bubble) is unrecoverable — the attribution model can never assign a bubble to a body that was never detected. In our case, this was the baseline YOLOv8s model.

4.4. Attribution: Candidate Pair Generation and Labels

On each comic page, we take every detected text bubble and match it with every detected body, creating a set of candidate pairs. For instance, if a page has 3 bubbles and 5 bodies, this produces 15 pairs. If the detected body overlaps the ground-truth speaker body at $\text{IoU} \geq 0.5$, the pair is labeled 1 (body is the speaker), otherwise 0 (body is not the speaker). During testing, for each text bubble, we run the classifier over all detected bodies on the page and assign the bubble to whichever body yielded the highest predicted probability.

4.5. Attribution: Algorithm & Baseline Comparisons

We train an XGBoost binary classifier [4] to score candidate speaker–bubble pairs. We select XGBoost for its built-in class imbalance handling, interpretable feature importance, and fast iteration across training configurations [3]. Key hyperparameters include `max_depth = 5`, `learning_rate = 0.05`, `subsample = 0.8`, `colsample_bytree = 0.8`, and the GPU histogram tree method. We set `scale_pos_weight = 14` to address the inherent class imbalance: each bubble produces exactly one positive pair (the true speaker) and several negative pairs (all other bodies on the page). Without this correction, the model collapses to predicting the majority class for nearly all candidate pairs. We also use early stopping with AUC as the evaluation metric, chosen for its stability under strong class imbalance compared to metrics like accuracy.

We compare the trained XGBoost classifier against three geometric baselines:

1. **Nearest centroid:** assigns each bubble to the body with the smallest centroid-to-centroid distance.
2. **Nearest edge:** assigns each bubble to the body with the smallest minimum edge-to-edge distance.
3. **Frame-aware:** assigns to the nearest-edge body within the same panel, falling back to nearest-edge globally if no body shares the panel.

4.6. Attribution: Training Data Construction

We evaluate two training regimes, distinguished by the bounding boxes used to compute features. The motivation is that at inference time, the attribution classifier only sees YOLO-predicted boxes, not ground-truth (GT) annotations. If the classifier is trained on GT coordinates, it learns from feature distributions that may differ systematically from inference: YOLO boxes can be slightly shifted, differently sized, or clipped at panel boundaries. Training on YOLO-matched coordinates closes this gap by exposing the classifier to the same imperfect boxes it will see at test time. We retain the GT regime as an upper bound, measuring how well attribution could perform given perfect detections.

In the GT regime, features are computed directly from Manga109’s ground-truth bounding box annotations. In the YOLO-matched regime, we run the fine-tuned detector on each training image, then compute features from YOLO-predicted boxes matched to ground-truth speaker–text pairs by maximum IoU. We set the matching threshold to be 0.3, which was chosen because the match only needs to identify which predicted box corresponds to each ground-truth annotation. A stricter cutoff (e.g., 0.5) would needlessly reduce the training set. Any unmatched pairs are discarded. In total, YOLO misses 7% of positive pairs across the 87 training volumes, leaving 1,958,846 training samples compared to 1,998,955 in the GT regime.

4.7. Attribution: Feature Engineering

For every candidate pair of text bubble and body, we build a feature vector of normalized geometric signals, described in Table 2. To maintain consistency across different page sizes in Manga109, all distance-based features are normalized by the image diagonal.

We additionally evaluate different versions of this feature set, either including or omitting the structural features, to understand each feature’s contribution under a controlled, isolated setting (Section 5.5):

- **v8** (8 features): omits `h_overlap` and `same_frame`
- **v9_no_frame** (9 features): omits `same_frame`
- **v9_no_overlap** (9 features): omits `h_overlap`
- **v10** (10 features): all features

4.8. Attribution: Sample Weighting Schemes

Not all attribution samples are equally difficult. Some text and body pairings are straightforward to match, while others are more challenging due to crowded scenes, partially occluded characters, or ambiguous panel layouts — precisely the cases where attribution errors are most likely and where additional training signal is most valuable.

To account for this unequal difficulty, we evaluate three sample weighting schemes:

- **Uniform**: all pairs are weighted equally ($w = 1$), serving as the standard empirical risk minimization baseline.

Table 2. Spatial features computed for each candidate (bubble, body) pair.

Category	Feature	Description
Proximity	<code>edge_dist</code>	Edge-to-edge Euclidean distance, normalised by page diagonal
	<code>centroid_dist</code>	Centroid-to-centroid distance, normalised by page diagonal
	<code>iou</code>	Intersection-over-union between bubble and body boxes
Structural	<code>same_frame</code>	1 if bubble and body centroids share the same panel frame, else 0
	<code>h_overlap</code>	Horizontal overlap normalised by bubble width
Relative Position	<code>rel_x</code>	Signed horizontal centroid offset, normalised by page width
	<code>rel_y</code>	Signed vertical centroid offset, normalised by page height
	<code>above_bonus</code>	1 if bubble centroid is above body centroid, else 0
Size	<code>text_area</code>	Bubble bounding box area as fraction of page area
	<code>body_area</code>	Body bounding box area as fraction of page area

- **Page-difficulty**: pairs from pages with more detected bodies receive higher weight (equal to the number of detected bodies on the page), on the reasoning that crowded pages present more distractors and are harder to attribute correctly.
- **YOLO-confidence**: for each pair, we match the ground-truth text bubble and body to their highest-IoU YOLO detections ($\text{IoU} \geq 0.3$) and compute $w = 1/\sqrt{\text{conf_text} \times \text{conf_body}}$, so that low-confidence (harder) detections receive higher weight. Unmatched boxes receive a neutral confidence of 0.5, and all weights are normalized to mean 1.0 before training.

4.9. Attribution: Evaluation Protocol

For each ground-truth text bubble and body pair in the evaluation set, we run the following pipeline:

1. Check whether any YOLO detection overlaps the ground-truth text box at $\text{IoU} \geq 0.5$. If not, mark the pair as *missed by YOLO* and skip.
2. For detected text bubbles, run the attribution model to predict a speaker.
3. Check whether the predicted body overlaps the ground-truth speaker at $\text{IoU} \geq 0.5$.

This pipeline yields three possible outcomes for every ground-truth pair: *missed by YOLO*, *incorrect speaker*, or *correct*. We report two metrics:

- **Overall accuracy:** correct/total
- **Conditional accuracy:** correct/detected, where detected = total – missed by YOLO

5. Experimental Results and Discussion

5.1. Detection: YOLO Architectural Changes

To evaluate whether targeted architectural modifications could improve detection on manga’s stylized imagery, we tested three YOLOv8s variants against the baseline, keeping all training hyperparameters identical. Table 3 summarizes the results.

The backbone modifications focusing on eliminating texture bias consistently underperformed. V1, which replaced BatchNorm with InstanceNorm in the stem and added a ViT self-attention block after SPPF, decreased mAP50-95 by 0.0068 while adding 3.18M parameters. V2, which introduced InstanceNorm and Squeeze-and-Excitation attention into every backbone C2f block, performed worst overall, dropping mAP50-95 by 0.0349. These negative results indicate that texture bias is not the detection bottleneck when the model is fine-tuned on manga.

A likely explanation for the failure of these texture-bias modifications is that the baseline YOLOv8s was already fine-tuned on Manga109’s stylized imagery. The pretrained weights provide general object detection features, and fine-tuning on 8,525 manga pages allows the model to adapt its feature representations to the manga domain directly, achieving the same domain transfer that the architectural modifications were designed to force structurally. In effect, fine-tuning renders the InstanceNorm and ViT modifications redundant. The model has already learned to handle manga’s visual style through training, so stripping texture information or imposing global attention provides no benefit and but instead removes useful learned cues. This suggests that texture-bias concerns from the cross-domain literature [1, 6] do not generalize to in-domain settings, and that training strategy should be prioritized over architectural complexity when the detector is fine-tuned on the target domain.

By contrast, V3 (CBAM Neck) was the only variant to improve over baseline across every metric, adding CBAM attention to the neck C2f blocks at layers. 20 epochs achieves a mAP50-95 of 0.6820, and at 40 epochs, this rises to 0.6864, and F1 improves from 0.8809 to 0.8857. The body class benefited more than text (+0.0064 AP50 vs −0.0004), consistent with body detection requiring stronger multi-scale feature selection due to the wide size variation of character bodies in manga. CBAM’s gains, while consistent, are minimal and do not affect attribution-stage conclusions.

Table 3. Architectural ablation on Manga109. CBAM neck modification only resulted in slight improvements over the baseline YOLOv8s model.

Variant	mAP50	mAP50-95	Prec.	Rec.	F1
YOLOv8s (baseline)	0.926	0.679	0.906	0.857	0.881
V1 Shape Backbone	0.922	0.672	0.906	0.851	0.878
V2 Style-Robust C2f	0.897	0.644	0.889	0.820	0.853
V3 CBAM Neck	0.928	0.686	0.908	0.864	0.886

5.2. Detection: XGBoosting

We trained an XGBoost binary classifier to score each YOLO detection as true positive or false positive based on 10 per-detection features (confidence, box dimensions, aspect ratio, position, class). Table 4 presents results at the optimal threshold of 0.2, selected from a sweep over {0.05, 0.1, 0.15, 0.2, 0.3, 0.4, 0.5, 0.6}. The filter removed 2,729 of 8,332 false positives (33%) while retaining 51,109 of 52,412 true positives (97.5%), raising precision from 0.863 to 0.901 and improving F1 from 0.873 to 0.880. Higher thresholds produced higher precision (up to 0.947) but at recall costs — at threshold 0.5, recall dropped to 0.799 and F1 fell below baseline.

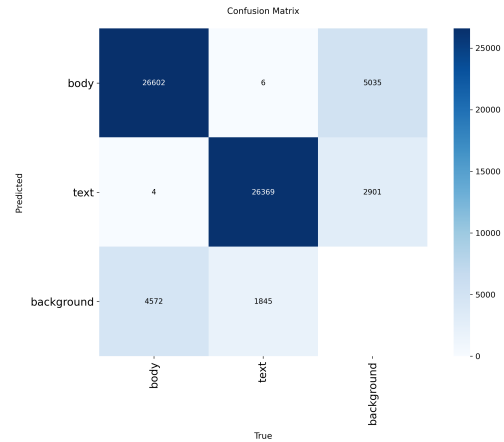


Figure 3. YOLOv8s confusion matrix. Note that the number of false positives (7936) is significantly greater than the number of false negatives (6417).

Table 4. XGBoost detection post-processing (threshold = 0.2) results in higher precision, but more false negatives than the original baseline YOLO.

System	Prec.	Rec.	F1	TP	FP	FN
YOLO only	0.863	0.882	0.873	52,412	8,332	6,986
YOLO + XGBoost	0.901	0.860	0.880	51,109	5,603	8,289

Despite the precision and F1 improvement from

XGBoost post-filtering, we use the unfiltered baseline YOLOv8s for the attribution pipeline. The reasoning lies in asymmetric error costs: a false negative (missed body or bubble) is unrecoverable — if YOLO does not detect a speaker’s body, the attribution classifier can never assign the bubble correctly. A false positive is recoverable — the attribution classifier will assign it a low spatial score and select a real body instead. The XGBoost filter improves precision by removing false positives but reduces recall by 2.2%; for attribution, that recall drop is more damaging than the precision gain.

5.3. Attribution: Geometric Baselines

Table 5 presents attribution results across all evaluated methods on the 22-volume validation set. The geometric baselines (nearest centroid and nearest edge) perform substantially worse than any trained classifier, confirming that attribution requires multiple spatial signals beyond proximity alone. The frame-aware baseline achieves 63.3%, a substantial 5.7 percentage point gain over nearest-edge and 7.7 percentage point gain over nearest-centroid, demonstrating that panel containment is a powerful structural prior for the problem of attribution.

Table 5. Geometric baselines vs. learned classifier on attribution.

Method	Acc.	Cond. Acc.	Wrong %	Δ vs. XGB
Nearest centroid	0.556	0.596	37.6%	−0.043
Nearest edge	0.576	0.618	35.6%	−0.023
Frame-aware	0.633	0.680	29.9%	+0.034
XGB, GT, uniform	0.599	0.643	33.3%	<i>baseline</i>

5.4. Attribution: GT Training vs. YOLO-Matched Training

We train XGBoost on both the GT and YOLO-matched training regimes using a uniform weighting scheme. As shown in Table 6, using YOLO-matched detections increases attribution accuracy by 5.2 percentage points, reaching 65.1%.

Table 6. GT vs. YOLO-matched training regimes (uniform weighting, v10 features).

Training Regime	Acc.	Cond. Acc.	Wrong %
XGB, GT coords	0.599	0.643	33.3%
XGB, YOLO-matched	0.651	0.698	28.1%

We attribute this gap to a distribution mismatch between training and inference. At test time, features are computed from YOLO detections rather than ground-truth coordinates. These detected boxes can be systematically different — slightly shifted, clipped at panel boundaries, or occasionally spurious. A classifier trained entirely in the GT

regime is therefore exposed to feature distributions at test time that differ from those seen during training, degrading performance.

5.5. Attribution: Feature Ablations

To isolate individual feature contributions, we train four variants on the YOLO-matched, confidence-weighted data, each including or omitting `same_frame` and `h_overlap` (Table 7).

Removing `same_frame` (while retaining all other features) drops accuracy by 1.91 percentage points — the largest single-feature impact observed. This demonstrates that `same_frame` provides an important training signal that proximity-based features cannot replicate. Even a precise edge-distance score remains ambiguous when two bodies are similarly close but lie across different panel boundaries.

Adding `h_overlap` on top of `same_frame` yields a small additional gain of 0.15 percentage points. This is consistent with `h_overlap` providing complementary signal — speech bubbles are often horizontally aligned with their speaker — though most of the structural information is already captured by `same_frame` alone.

Interestingly, the 8-feature baseline (no structural features) still reaches 63.1% accuracy, nearly matching the frame-aware geometric baseline of 63.3%. This suggests the learned model can implicitly recover much of the panel-containment signal from proximity features alone. However, the cleanest and most consistent gains occur only when `same_frame` is combined with the full proximity-based feature set.

Table 7. Feature set ablation (YOLO-matched, conf-weighted).

Feature Set	# Feats	Acc.	Cond. Acc.
No <code>same_frame</code> , no <code>h_overlap</code> (v8)	8	0.631	0.676
No <code>same_frame</code> , + <code>h_overlap</code> (v9)	9	0.632	0.678
<code>same_frame</code> , no <code>h_overlap</code> (v9)	9	0.650	0.697
<code>same_frame</code> + <code>h_overlap</code> ★ (v10)	10	0.651	0.699

5.6. Attribution: Weight Scheme Ablations

Across the different weighing schemes used for the YOLO-matched training regime, the differences in performance are not very large, with accuracy varying by at most 0.15 percentage points within each feature group (Table 8). On the other hand, the contrast across the weighting scheme for the GT training regime is slightly larger. Under the GT regime, confidence weighting yielded a 0.39 percentage point improvement over the uniform weighting. This larger margin seen is likely due to the fact that weighting partially compensates for the distributional mismatch. Since the model is trained on ground-truth coordinates but evaluated on YOLO

detections, emphasizing harder or lower-confidence examples may help the classifier adapt to feature patterns that more closely resemble inference-time conditions.

In the YOLO-matched regime, training and test-time distributions are already aligned by construction. As a result, the benefit of weighting largely disappears, since there is less mismatch for the weighting scheme to compensate for. These results suggest that, in Detect-then-Match pipelines, maintaining alignment between training and inference distributions is substantially more important than fine-tuning the weighting scheme itself.

Table 8. Comparison of three sample-weighting strategies across the full v10 feature set under GT and YOLO-matched training. Val AUC measured on internal 80/20 split of the 87 training volumes.

Regime	Weights	Val AUC	Acc.	Cond. Acc.
GT coords	Uniform	0.9645	0.599	0.643
GT coords	Page-diff	0.9645	0.596	0.640
GT coords	YOLO-conf	0.9645	0.603	0.647
YOLO-matched	Uniform	0.9641	0.651	0.698
YOLO-matched	Page-diff	0.9639	0.651	0.698
YOLO-matched	YOLO-conf ★	0.9640	0.651	0.699

5.7. Attribution: Feature Importance

On our best model, XGBoost under a YOLO-matched training regime, using all 10 features and a confidence weighting scheme, achieves a 65.1% accuracy and 69.9% conditional accuracy on the 22 held-out validation volumes. This 4.8 percentage point gap between accuracy and conditional accuracy quantifies the cost of YOLO’s 6.8% missed detection rate on the end-to-end pipeline. This model was able to outperform the nearest-edge baseline by 7.6 percentage points, the frame-aware baseline by 1.8 percentage points, and the best GT-trained regime by 4.8 percentage points.

Examining the most important features of the model in Figure 4, we see that in the XGBoost model trained on the full 10-feature set, `same_frame` is by far the most dominant feature at a 0.82 gain, which is much higher than the next most important feature, `edge_dist`, with a gain of 0.09. This is because `same_frame` often serves as a near-perfect discriminator for a large portion of the pages. When a text bubble and a speaker share the same panel, the model can often resolve attribution confidently without any need to further examine other proximity features. These other proximity-based features are only called upon for harder cases where `same_frame` is not informative, such as when there are multiple modifies within the same panel.

If the XGBoost model is trained without `same_frame`, the ranking shifts. `Edge_dist` and `centroid_dist` are now the top 2 most important features, but their relative importance to the 8-feature model has dramatically increased. `edge_dist` now has a 0.60 gain, carrying a lot of the

discriminative load that `same_frame` previously carried. This is likely because the model had no choice but to rely on it, and not because of its usefulness in isolation. In addition, `edge_dist` and `same_frame` are rather closely correlated features (characters close to a text bubble also tend to share the same panel). These two features are partially redundant with each other, and there is a suppression effect going on where whichever is present first silences the apparent importance of the other. This effect also explains why removing `same_frame` only costs around 1.9% in accuracy, as `edge_dist` is able to compensate and provide enough signal to the model to still discriminate.

5.8. Limitations and Future Work

5.8.1. Dataset Limitations

Our evaluation is conducted entirely on Manga109, a dataset of Japanese manga. However, western comics, webtoons, and manhwa have distinct visual conventions for character-bubble placement, panel layout, and artistic style, and performance on these formats is unknown. Future work should incorporate comics of various art styles and origins.

5.8.2. YOLO Model Limitations

On the detection side, our experiments were constrained by compute resources. Most models were trained for only 20 epochs, and we only evaluated the small variant of YOLOv8. Larger variants (YOLOv8m, YOLOv8l) have substantially more capacity and would likely improve detection metrics. Furthermore, we only detect two classes (body and text). In the future, incorporating face and panel frame detection into the YOLO model could provide additional spatial signals, for example, face bounding boxes are often closer to speech bubbles than full body boxes, and predicted panel boundaries would reduce the pipeline’s dependence on ground-truth annotations.

5.8.3. Attribution Pipeline Limitations

Our best model reaches 65.1% attribution accuracy — roughly two out of three bubbles correctly attributed — which beats nearest-edge (57.6%) but falls short of Li et al. [14], who report 70–77% using graph-based models with richer representations. Of the 25,475 evaluated pairs, 1,731 (6.8%) were failure cases due to YOLO detection misses and are unrecoverable regardless of attribution quality. The remaining 28% wrong attributions might come down to geometry not being enough: the model has no access to bubble tail direction, character gaze, or reading order, which are the cues a human read would use when multiple characters are equally close to a bubble.

6. Ethical Concerns and Implications

This work raises several ethical considerations that must be addressed to mitigate any potential negative impacts.

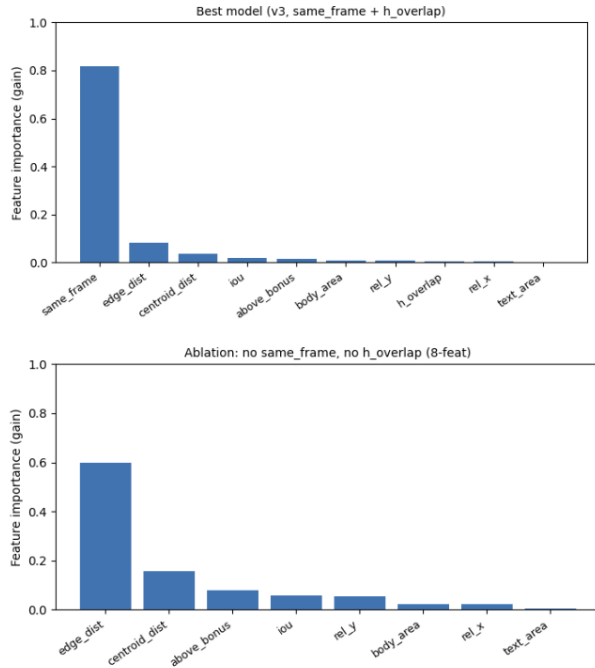


Figure 4. Feature importance graphs for the XGBoost classifier. Top: Best-performing model, using the YOLO-matched training regime, confidence weighting, and all 10 features. Bottom: Ablation with same_frame and h_overlap removed (8 features).

For one, manga is protected intellectual property. Computer vision models that automatically parse manga volumes for attribution could make it easier to extract and redistribute copyrighted material without permission. Researchers should broadcast this risk and avoid distributing tools to facilitate copyright infringement.

Another ethical consideration is that manga datasets may represent a limited range of artistic styles, which could lead to deception and attribution models that perform poorly on underrepresented styles. Researchers should be transparent about the limitations of their datasets and be specific about what art styles their tools perform best on.

Furthermore, this work could also have negative implications for the art industry. Automated attribution could reduce the demand for manual annotation in manga digitization and translation workflows, which is important work currently performed by skilled human editors. While such technology can accelerate workflows, it also risks displacing these professionals. To mitigate against these adverse effects, researchers should advocate for workflows in which automated tools support and augment human editors rather than replace them.

7. Conclusion

We propose a Detect-then-Match pipeline for manga speech bubble attribution, combining YOLOv8 detection with an XGBoost spatial classifier on Manga109. Our detection ablation shows that backbone modifications targeting texture bias are redundant when the model is already fine-tuned on manga, while CBAM neck attention provides the only consistent improvement. For attribution, aligning training features with YOLO predictions rather than ground-truth boxes yields the largest accuracy gain, and panel co-membership emerges as the dominant feature at 82% importance. Our best model achieves 65.1% accuracy on 22 volumes, outperforming geometric baselines by 7.6%. These results demonstrate that relatively simple spatial features can capture a meaningful share of the visual cues in manga to encode speaker identity, providing promising implications for downstream tasks like automated translation, audio-described reading, and character-aware search over large manga collections.

8. AI Disclaimer, Partner Contributions, Codebase

8.1. AI and Research

Claude was used to help identify papers relevant to our topic, as it is so broad and well-researched.

8.2. AI and Code

We also used Claude to help code our model, specifically helping to substitute the new modules into YOLOv8 via PatchedTrainer, write evaluation code, and to help us debug pipeline issues as many things did not integrate properly at first. Additionally, AI helped us implement the XGBoosting strategy.

8.3. AI and Formatting

Claude was additionally used to generate LaTeX versions of the tables in this paper, as well as making sure all of our google docs formatting transferred over to overleaf. Chat GPT was used to create the bibliography.

8.4. Partner Contributions

Research was split equally amongst the group. Ruoxi focused more on implementing the detection stage of the pipeline, while Minnie focused more on implementing the attribution stage. The final report was a collaborative effort.

8.5. Codebase

Code for this project has been made publicly available, and can be viewed here: https://drive.google.com/drive/folders/1TNDU3FVwYcpQRTxb8xpAEYQ8Fyse_RUY?usp=sharing

References

- [1] T. Ahmad et al. Toward cross-domain object detection in artwork images using improved yolov5 and xgboosting. *IET Image Processing*, 17(10):2925–2939, 2023. 1, 2, 4, 6
- [2] Kiyoharu Aizawa, Azusa Fujimoto, Akinori Otsubo, Takahiro Ogawa, Yusuke Matsui, Kenta Tsubota, and Hitoshi Ikuta. Building a manga dataset “manga109” with annotations for multimedia applications. *IEEE MultiMedia*, 27(2):8–18, 2020. 1
- [3] Amit R. Bhende. A robust XGBoost approach for bearing fault prediction from vibration signals: Feature analysis and model evaluation. *Noise & Vibration Worldwide*, 2025. 4
- [4] Tianqi Chen and Carlos Guestrin. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, page 785–794. ACM, 2016. 4
- [5] Azuma Fujimoto, Toru Ogawa, Kazuyoshi Yamamoto, Yusuke Matsui, Toshihiko Yamasaki, and Kiyoharu Aizawa. Manga109 dataset and creation of metadata. In *Proceedings of the 1st International Workshop on coMics ANalysis, Processing and Understanding (MANPU)*, 2016. 2
- [6] Robert Geirhos, Patricia Rubisch, Claudio Michaelis, Matthias Bethge, Felix A. Wichmann, and Wieland Brendel. Imagenet-trained CNNs are biased towards texture; increasing shape bias improves accuracy and robustness. In *International Conference on Learning Representations*, 2019. 2, 6
- [7] Jie Hu, Li Shen, and Gang Sun. Squeeze-and-excitation networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 7132–7141, 2018. 2, 4
- [8] Xun Huang and Serge Belongie. Arbitrary style transfer in real-time with adaptive instance normalization. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 1501–1510, 2017. 2, 3
- [9] Glenn Jocher and Jing Qiu. Ultralytics yolo26. <https://github.com/ultralytics/ultralytics>, 2026. 1
- [10] R. Khanam and M. Hussain. Yolov11: An overview of the key architectural enhancements. *arXiv preprint arXiv:2410.17725*, 2024. 1
- [11] Jochen Laubrock and Alexander Dunst. *Computational Approaches to Comics Analysis*. Bloomsbury Academic, 2019. 2
- [12] Y. Li, K. Aizawa, and Y. Matsui. Manga109dialog: A large-scale dialogue dataset for comics speaker detection. In *ICME*, 2024. 1
- [13] Yingxuan Li, Kiyoharu Aizawa, and Yusuke Matsui. Manga109dialog: A large-scale dialogue dataset for comics speaker detection, 2024. 2
- [14] Y. Li, R. Hinami, K. Aizawa, and Y. Matsui. Zero-shot character identification and speaker prediction in comics via iterative multimodal fusion. In *ACM Multimedia*, 2024. 1, 8
- [15] Zhihong Liu, Jianji Wang, Hui Chen, Yongqiang Ma, and Nanning Zheng. Relation-specific feature augmentation for unbiased scene graph generation. *Pattern Recognition*, 157: 110936, 2025. 2
- [16] N. V. Nguyen, C. Rigaud, and J.-C. Burie. Digital comics image indexing based on deep learning. *Journal of Imaging*, 4(7):89, 2018. 1
- [17] Nhu-Van Nguyen, Christophe Rigaud, and Jean-Christophe Burie. Comic MTL: optimized multi-task learning for comic book image analysis. *International Journal on Document Analysis and Recognition (IJDAR)*, 22(3):265–284, 2019. 2
- [18] Takahiro Ogawa, Akinori Otsubo, Ryo Narita, Yusuke Matsui, Toshihiko Yamasaki, and Kiyoharu Aizawa. Object detection for comics using manga109 annotations. *arXiv preprint arXiv:1803.08670*, 2018. 1
- [19] Christophe Rigaud, Norbert Tsopze, Jean-Christophe Burie, and Jean-Marc Ogier. Robust frame and text extraction from comic books. In *International Workshop on Graphics Recognition (GREC)*, 2013. 2
- [20] Christophe Rigaud, Nhu-Linh Thanh, Jean-Christophe Burie, Jean-Marc Ogier, Motoi Iwata, Eiki Imazu, and Koichi Kise. Speech balloon and speaker association for comics and manga understanding. In *ICDAR*, pages 351–355, 2015. 1
- [21] Ragav Sachdeva and Andrew Zisserman. The manga whisperer: Automatically generating transcriptions for comics. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 12967–12976, 2024. 2
- [22] Sanghyun Woo, Jongchan Park, Joon-Young Lee, and In So Kweon. CBAM: Convolutional block attention module. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 3–19, 2018. 2